

컴시 수행평가

선점형 스케줄링(preemptive scheduling)

운영체제가 필요하다고 판단하면 실행 상태에 있는 프로세스의 작업을 중단시키고 새로운 작업을 시작할 수 있는 방식

대부분의 저수준 스케줄러는 선점형 스케줄링 방식을 사용

장점

1. 응답 시간 향상 : 짧은 작업이나 새로운 작업이 빨리 실행될 수 있어, 사용자 체감 속도가 좋아짐
2. CPU 지원 분배가 공정 : 사용자가 사용중인 모든 작업이 잘 돌아가니까 체감 성능이 좋다. (여러 앱을 사용해도 잘 실행됨)
3. 우선순위 기반이기 때문에 긴급한 시스템 요청이나 중요한 프로세스를 빠르게 처리 가능



단점

- ▼ 1 문맥 교환의 오버헤드(실제 작업 이외에 추가로 해야 하는 관리 작업이 나 자원 낭비, 성능 손실 등)가 많다.

오버헤드가 많아질수록 실제 작업에 사용할 자원이 줄어들고, 시스템이 느려지거나 비효율적이 될 수 있다.

오버헤드가 발생하는 이유 : CPU는 동시에 여러 프로세스를 처리하지 못하기 때문에 프로세스 간을 왔다 갔다 하면서 처리하는데 이때 이전 프로세스의 상태를 저장하고 다음 프로세스의 상태를 불러오는 과정에서 생긴다.

2. 우선순위가 높은 프로세스가 CPU를 점유하는 방식으로 반환 시간 의 예측이 어렵다.

선점 스케줄링의 종류

1. 라운드 로빈 (Round Robin, RR)

- 시간 할당량을 정해 놓고, 그 시간이 지나면 무조건 다음 프로세스로 전환
- 간단하고 공정하지만, 시간 할당량 설정이 매우 중요

2. 최소잔류시간우선 스케줄링 (SRTF, Shortest Remaining Time First)

- 남은 실행 시간이 가장 짧은 작업을 우선 실행
- 실행 중인 프로세스보다 더 짧은 남은 시간을 가진 프로세스가 도착하면 선점

3. 우선순위 기반 스케줄링(Preemptive Priority Scheduling)

- 우선순위가 높은 프로세스가 도착하면, 현재 실행 중인 프로세스를 강제로 중단하고 실행
- 실시간 처리나 시스템 자원 제어에 유리

4. 다단계 큐 스케줄링 (MLQ, Multilevel Queue Scheduling)

- 프로세스를 우선순위나 특성에 따라 여러 큐로 분리
- 각 큐마다 다른 스케줄링 방식 사용 가능 (RR, SRTF 등)
- 큐 간 이동이 불가능해서 하위 단계의 큐는
- 큐 이름 | 대상 프로세스 | 사용 스케줄링 알고리즘 | 이유
- 인터랙티브 큐 | 사용자 입력, GUI, 터미널 등 | RR | 빠른 응답이 중요하고 짧은 시간 할당으로 빠르게 전환
- 시스템 큐 | OS 서비스, 커널 작업 | 우선순위 기반 스케줄링 | 중요한 작업이므로 가장 빠르게 처리
- 배치 큐 | 긴 시간 걸리는 계산, 백업 등 | FCFS 또는 SRTF | 효율적으로 CPU 사용, 응답 속도보다 처리량 중시
- 백그라운드 큐 | 자동 업데이트, 로그 저장 | FCFS 또는 긴 RR | 응답은 느려도 되고 시스템 리소스를 적게 써도 됨

5. 다단계 피드백 큐 (MFQ, Multilevel Feedback Queue)

- 다단계 큐 + 프로세스의 행동에 따라 큐 간 이동이 가능
- 짧은 작업은 높은 우선순위 큐에서 빨리 실행되고, 긴 작업은 점차 낮은 우선순위 큐로 이동
- 유연하면서 효율적인 선점형 스케줄링 (기아 현상 예방 가능)